Project Synopsis

Title: AUTOSAR HLD AI Assistant: Grounded AI for High-Level Design Analysis, Engineering Q&A and Architecture Exploration

Abstract

Automotive AUTOSAR-style High-Level Design (HLD) documents contain descriptions of Software Components (SWCs), ports, interfaces, ECU mappings and communication paths. Manual searches are time-consuming, while conventional LLM answers may fabricate technically plausible details absent from a source specification. This capstone builds a document-grounded engineering assistant combining PDF extraction, semantic search, retrieval-augmented generation (RAG), citation ID validation and structured AUTOSAR entity exploration.

PyMuPDF extracts page/block text and layout metadata. Text cleaning and hierarchical chunking prepare document passages for a pretrained all-MiniLM-L6-v2 sentence encoder. Embeddings and source metadata are indexed in ChromaDB. At query time, vector search retrieves relevant chunks, which a grounded prompt passes to Gemini or a deterministic MockLLM provider. A citation verifier confirms that cited chunk IDs refer to retrieved evidence. Separately, a rules/table-based extractor captures Software Components, Ports and Interfaces in SQLite. A FastAPI service and Streamlit workbench expose upload, engineering Q&A, source snippets and entity exploration.

Problem and intended users

HLD information is spread across tables and sections, making manual tracing difficult. Exact keyword searching misses paraphrased questions. An unrestricted generative model can invent unsupported automotive facts. The intended users are automotive software engineers, systems architects, reviewers, integration engineers and educational evaluators. The tool supports human review; it is not a safety-certification application.

Objectives

1. Extract/clean PDF engineering text while preserving source provenance.
2. Split it into structurally meaningful chunks and embed with a pretrained transformer model.
3. Build semantic retrieval with ChromaDB and metadata-based source links.
4. Generate evidence-constrained answers and explicit insufficient-evidence responses.
5. Verify citation identifiers against retrieved chunks and expose snippets.
6. Extract SWC/Port/Interface entities into structured SQLite records.
7. Provide an interactive FastAPI + Streamlit implementation and automated tests.

Solution and methodology

The indexing path is: PDF → PyMuPDF → text cleaning → structural chunking → MiniLM embeddings → ChromaDB. The question path is: user question → embedding → top-k retrieval

→ context- and citation-constrained prompt → Gemini/MockLLM → citation ID validator → UI. The architecture path uses deterministic regex/table extraction and SQLite for structured entity browsing.

Technology

Python, PyMuPDF, Pydantic, Sentence Transformers (all-MiniLM-L6-v2), ChromaDB, Gemini API, MockLLM, SQLite, FastAPI, Uvicorn, Streamlit and pytest.

AI contribution and innovations

The AI parts are pretrained semantic text representation and context-constrained natural-language answer generation; no foundation model was trained and no PCA/UMAP/t-SNE was added. Distinctive contributions are source-level metadata/citation checks, a domain-specific AUTOSAR entity explorer, two fit-for-purpose databases and provider-switchable offline testing.

Evaluation and limitations

The repository includes a ten-question braking-system golden dataset and tests for ingestion, retrieval, prompting, citation IDs, extraction, APIs and UI. Previous student-reported results (70 passed, 1 skipped) are historical, not a fresh run. Manual tests have shown generic MockLLM answers with valid chunk IDs, including an insufficient-evidence failure; a valid citation ID is not evidence of semantic factual accuracy. Gemini evaluation must therefore compare each answer with its source. The prototype primarily handles text PDFs; difficult tables, scanned PDFs, exact identifier search and compliance assurance are future work.

Future enhancements

Hybrid BM25 + vector search, richer graph relationships, ARXML ingest, OCR/table improvements, claim-level entailment checks, systematic verified benchmarks and access controls.

Conclusion

The working prototype combines semantic document intelligence with domain-specific AUTOSAR architecture extraction and traceable evidence display. It helps engineers locate relevant design facts while preserving their responsibility to validate engineering conclusions.